

Yes. If you want to **start building this right now**, I would not try to learn “everything about AI agents” first. You need a fairly small set of skills, and you can learn each one **while building the project**.

Your end product is:

> **A local AI agent that researches the correct GCSE specification, reads legitimate study resources, creates syllabus-mapped flashcards, checks them, detects duplicates, and adds/updates cards in the correct Anki deck.**

1. What you actually need to learn

Level 1 — Python fundamentals ■■■■■■

You don't need advanced Python.

Learn:

- Variables
- Strings/numbers/lists/dictionaries
- ``if`` statements
- ``for`/`while`` loops
- Functions
- Classes — basic understanding
- Imports/modules
- Exceptions (``try`/`except``)
- Reading/writing files
- JSON
- Virtual environments
- Installing packages with ``pip`/`uv``
- Environment variables

The official Python tutorial covers the language fundamentals and standard library.
([docs.python.org](https://docs.python.org/3/?utm_source=chatgpt.com))

Target: You should be able to look at Python code generated by an AI and understand roughly what every section is doing.

You don't need to memorise Python syntax.

2. APIs ■■■■■■

This is **really important** for your project.

You need to understand:

Your program
↓
HTTP request
↓
API
↓
JSON response

For example:

Python
↓
"Give me my Anki decks"
↓
AnkiConnect
↓
["GCSE", "GCSE::Chemistry", ...]

Learn:

- HTTP
- GET / POST
- JSON
- API endpoints
- Request/response
- Authentication
- API keys
- Error responses

You don't need to become a web developer.

3. AnkiConnect ■■■■■

This is the first integration I'd actually build.

AnkiConnect exposes Anki's functionality through an HTTP API, normally on port `8765`. It supports actions such as retrieving decks and adding notes.

([github.com](https://github.com/ankiultimate/anki-connect?utm_source=chatgpt.com))

Learn how to:

getDeckNames
getModels
modelFieldNames
findNotes
notesInfo
addNote
addNotes

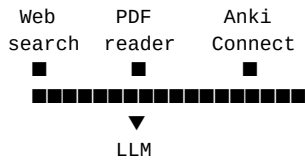
Your first milestone should be:

> **Python** → **AnkiConnect** → **create one test card**

...

— — —

[illegible]



The model decides which tool it needs.

Your Python application defines what those tools can do.

You **don't need LangChain or another agent framework immediately**. I'd build the first version yourself so you understand what's actually happening.

6. Web research

Your agent needs to distinguish:

Good source:

> Official CAIE syllabus

from:

Potentially unreliable source:

> Random revision website

Build source priorities into the agent:

```

Official exam board
↓
Official educational material
↓
Legitimate textbook/open resource
↓
Reputable educational website
↓
Other sources
  
```

And make the agent record:

```

source
URL
date accessed
syllabus version
  
```

That will become very useful when specifications change.

7. PDF/document processing

You need to learn how your program can turn:

```
PDF
↓
Text
↓
AI
```

rather than trying to send huge textbooks blindly to the model.

Learn:

- PDF text extraction
- Page/chapter detection
- Chunking
- OCR for scanned PDFs
- Token/context limits

Eventually:

```
500-page textbook
↓
Chapter 6
↓
Relevant pages
↓
Chunks
↓
AI
```

8. Databases — later

You don't need a complicated database initially.

Start with **SQLite**.

Store things like:

```
processed_files
syllabus_versions
generated_cards
anki_note_ids
processing_history
```

This allows the agent to know:

> "I've already processed this chapter."

and:

> "These 43 cards came from this syllabus version."

9. Basic software engineering ■■■■

You need enough to make the system reliable.

Learn:

- Git/GitHub
- `.env` files
- API-key security
- Logging
- Error handling
- Testing
- Project structure
- Configuration files

This is important because you'll eventually have:

```
agent/  
■■■ agent.py  
■■■ anki.py  
■■■ web.py  
■■■ pdf.py  
■■■ syllabus.py  
■■■ cards.py  
■■■ database.py
```

rather than one 3,000-line file.

10. You DON'T need to learn these yet

Don't waste time learning:

- Machine-learning mathematics
- Training neural networks
- Building an LLM
- CUDA
- TensorFlow/PyTorch
- Kubernetes
- Advanced cloud architecture
- Advanced LangChain
- Multi-agent systems

Your project is primarily **AI application engineering**, not AI model research.

What you need to install

I'd use **Windows + Antigravity + Python** since you're already interested in Google Antigravity.

Required

- **Python**
- **Google Antigravity**
- **Anki**
- **AnkiConnect**
- **Git**
- A code editor/IDE (Antigravity IDE can fill this role)
- An LLM API account/API key

Google's current Antigravity documentation says it supports Windows and provides an IDE, agent manager and CLI, so it is suitable for developing this locally. ([codelabs.developers.google.com](https://codelabs.developers.google.com/getting-started-google-antigravity?utm_source=chatgpt.com))

Later

- SQLite — already included with Python
- FastAPI — if you want a proper backend/API
- PDF-processing libraries
- OCR library
- Web-search API
- Possibly an external tunnel if you eventually need remote access

You **do not need FastAPI on day one**. FastAPI becomes useful when you want a proper API or web interface; its official tutorial is a good reference when you reach that stage.

([fastapi.tiangolo.com](https://fastapi.tiangolo.com/tutorial/?utm_source=chatgpt.com))

Your learning + building roadmap

I'd do this over roughly **8 stages**.

Stage 1 — Python

Learn enough Python to make:

```
input
↓
function
↓
process
↓
output
```

Build tiny projects:

- Calculator
- File reader
- JSON reader/writer
- PDF text extractor
- Simple API caller

Don't spend months learning Python before starting the project.

Stage 2 — Anki connection

Forget AI.

Make:

```
Python
↓
AnkiConnect
↓
Anki
```

Your first program should:

1. Connect to Anki.
2. Get all deck names.
3. Print them.
4. Create one test card.
5. Find that card.
6. Delete it.

Once you can do that, you've learned APIs + AnkiConnect.

The AnkiConnect GitHub documentation is the key reference.

([github.com](https://github.com/ankiultimate/anki-connect))(https://github.com/ankiultimate/anki-connect?utm_source=chatgpt.com))

Stage 3 — AI connection

Make:

```
Python
↓
LLM
↓
"Hello"
```


Then:

```
Python
↓
LLM
↓
Generate 5 flashcards
↓
JSON
```

Don't connect Anki yet.

Stage 4 — Build your first actual agent

Give your LLM three tools:

```
search_web()
read_file()
get_anki_decks()
```

Then give it the goal:

> Research this GCSE topic and determine what information is required.

Now you have something genuinely agentic.

Stage 5 — Add syllabus research

Now implement:

```
Subject
↓
Exam board
↓
Current specification
↓
Relevant topic
↓
Learning objectives
```

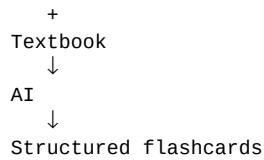
Make the agent **store the specification version** it used.

This is crucial because GCSE specifications can change.

Stage 6 — Add flashcard generation

Now:

```
Syllabus
```

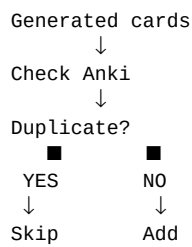


Add your rules:

- One idea per card
- Exam-focused
- No unnecessary cards
- No practice questions
- Correct terminology
- No duplicates
- Cover syllabus requirements

Stage 7 — Add the Anki agent

Now combine:



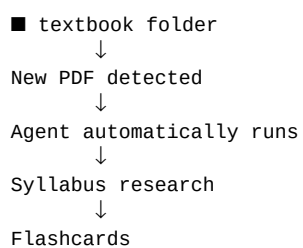
Then automatically choose:

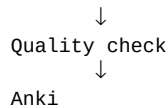
```
Chemistry → GCSE::Chemistry
Biology → GCSE::Biology
Physics → GCSE::Physics
```

Stage 8 — Make it autonomous

Only **after everything works manually**.

Then add:





That's when you've reached the system you've been imagining.

Resources I'd use

■ Python

Official Python tutorial — use this as your reference rather than trying to memorise everything.
([docs.python.org](https://docs.python.org/3/tutorial/index.html?gdm=GetApp&utm_source=chatgpt.com))

[Python official tutorial](https://docs.python.org/3/tutorial/?utm_source=chatgpt.com)

For YouTube, good beginner-friendly channels include:

- **freeCodeCamp**
- **Corey Schafer**
- **Programming with Mosh**

Don't watch an entire 20-hour course before coding. Watch a section → build something → continue.

■ Google Antigravity

This is particularly relevant because you're considering using it as your AI coding partner.

Google's official codelab covers installation, projects, agents, permissions and workflows. ([codelabs.developers.google.com](https://codelabs.developers.google.com/getting-started-google-antigravity?utm_source=chatgpt.com))

[Google Antigravity — Getting Started](https://codelabs.developers.google.com/getting-started-google-antigravity?utm_source=chatgpt.com)

There's also an official **Building with Google Antigravity** codelab that goes into building actual applications with agents. ([codelabs.developers.google.com](https://codelabs.developers.google.com/building-with-google-antigravity?authuser=0&hl=en&utm_source=chatgpt.com))

[Google Antigravity — Building with Antigravity](https://codelabs.developers.google.com/building-with-google-antigravity?utm_source=chatgpt.com)

Google also has an official Antigravity YouTube channel. ([codelabs.developers.google.com](https://codelabs.developers.google.com/getting-started-google-antigravity?utm_source=chatgpt.com))

[Google Antigravity YouTube](https://www.youtube.com/@googleantigravity?utm_source=chatgpt.com)

■■ AnkiConnect

This is one of your **most important resources**.

[AnkiConnect GitHub/documentation](https://github.com/ankiultimate/anki-connect?utm_source=chatgpt.com)

Read the API actions rather than trying to understand the entire repository.

Focus on:

```
deckNames
modelName
modelFieldNames
findNotes
notesInfo
addNote
addNotes
updateNoteFields
```

■ FastAPI

Don't learn this immediately.

When you're ready to build your interface/backend:

[FastAPI official tutorial](https://fastapi.tiangolo.com/tutorial/?utm_source=chatgpt.com)

The official tutorial is structured so you can work through it incrementally.

([fastapi.tiangolo.com](https://fastapi.tiangolo.com/tutorial/?utm_source=chatgpt.com))

Your first week

If I were you, I'd do this:

Day 1

Python basics

Learn:

- Variables
- Lists
- Dictionaries
- Functions
- Loops
- `if`
- imports

Build a tiny program.

Day 2

JSON + APIs

Learn:

HTTP
JSON
POST
GET
API

Make Python call a simple public API.

Day 3

AnkiConnect

Install it.

Make Python:

connect → get decks → print decks

Day 4

Make Python:

connect → create card → Anki

This is your first major milestone.

Day 5

LLM API

Make:

Python → AI → JSON flashcards

Day 6

Combine them:

```
Python
↓
AI generates cards
↓
Python validates JSON
↓
AnkiConnect
↓
Anki
```

You now have a **very primitive version of your agent**.

Day 7

Add:

```
Web search
↓
CAIE syllabus
↓
AI
↓
Flashcards
↓
Anki
```

And you've officially built the core of your project.

One rule I strongly recommend

Don't ask Antigravity to build the entire thing immediately.

Instead, tell it:

> "Build Phase 1 only. Connect Python to AnkiConnect and implement getDeckNames. Do not implement anything else."

Test it.

Then:

> "Now implement addNote."

Test it.

Then:

> "Now add the LLM."

This makes debugging **dramatically easier**, and you'll actually understand what you've built.

If you want to start **right now**, I'd make your first milestone:

> **Python** → **AnkiConnect** → **list my Anki decks**

Once that works, we can build the next component on top of it.